COSC264 · INTRODUCTION TO COMPUTER NETWORKS AND THE INTERNET · TERM 4 · MODULE 6

Application Protocols: the Web, Email and DNS

1 · Network applications | 2 · The Web and HTTP | 3 · Email | 4 · DNS

cosc264.up.railway.app

OUTLINE

Module 6 — roadmap

General concepts about network applications

One network application: the Web

Web page content layout, URL addressing

HTTP: the protocol that supports the Web

Connections, messages, cookies, web caching

Email

SMTP, POP3 and IMAP · message format and MIME

DNS: the Domain Name System

Hierarchy, caching, resource records

MODULE 6

Network Applications

What an application needs from the layers beneath it

1 · Network applications | 2 · The Web and HTTP | 3 · Email | 4 · DNS

NETWORK APPLICATIONS

What is a network application?

Programs that run on different end systems and communicate with each other over the network.

Web browsing

Google Chrome, Mozilla Firefox, Microsoft Edge

Email

Gmail, Outlook

Remote access

PuTTY, Remote Desktop

Instant messaging

Slack, Microsoft Teams

Streaming services

Netflix, Spotify, YouTube

File transfer

FileZilla, WinSCP

What applications need from the transport layer

- Application-layer protocols are controlled by the **application developer**; transport-layer protocols, however, are built into the **OS**.
- As a developer you can only choose between the protocols the OS provides — so the choice matters.

Property	Requires it	Can do without	Transport protocol
Reliable data transfer	Email, IM, file transfer, financial apps	Multimedia apps (loss-tolerant)	**TCP** preferred UDP unsuitable
High bandwidth	Internet telephony, multimedia apps	Email, file transfer (elastic)	**UDP** preferred TCP throttles itself
Low delay	Internet telephony, teleconferencing, multiplayer games	Email, file transfer (delay-tolerant)	**UDP** preferred TCP retransmissions add latency

NETWORK APPLICATIONS

Application-layer protocols

An application-layer protocol defines how an application's processes, running on different end systems, **pass messages to each other**.

An application-layer protocol is **only one piece** (a big piece) of a network application.

Network application	Application protocol
Web application	HTTP
Email	SMTP

MODULE 6

The Web and HTTP

The application protocol behind every page you load

1 · Network applications | **2 · The Web and HTTP** | 3 · Email | 4 · DNS

The World Wide Web and HTTP

THE WEB

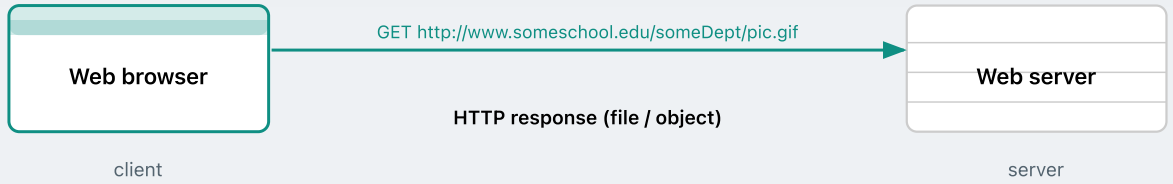
The **World Wide Web** (aka Web) is a global collection of documents and other resources to be shared over the Internet — like a large file storage system. Anyone can request a file and get it back.

Each object on the Web (e.g., an image, an HTML file) is addressable by a **URL**:

```
http://www.someschool.edu/someDept/pic.gif
      └─ host name ─┘ └──── path name ────┘
```

HTTP

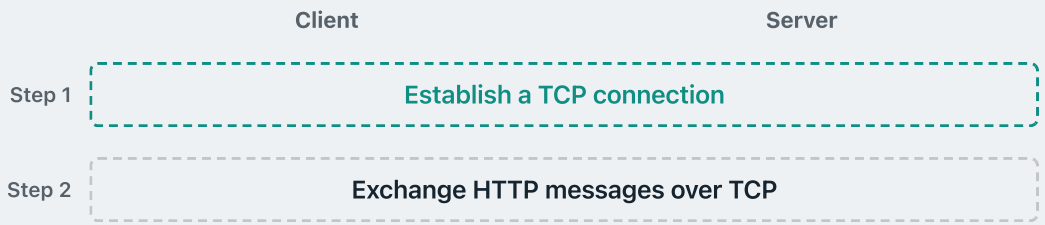
The **Hypertext Transfer Protocol** is the application-layer protocol that supports the Web — it defines how clients request objects by URL and how servers respond.



HTTP

How HTTP works

HTTP uses **TCP** as its transport protocol — so before any HTTP message is sent, a TCP connection must be established first. (HTTP/3 uses QUIC instead — an exception.)



STATE LESS

HTTP is **stateless** — the server maintains no information about past client requests. Each request is handled independently.

HTTP connections

Non-persistent HTTP

- At most **one object** is sent over a TCP connection — connection is then closed
- HTTP/1.0 uses non-persistent HTTP

Persistent HTTP

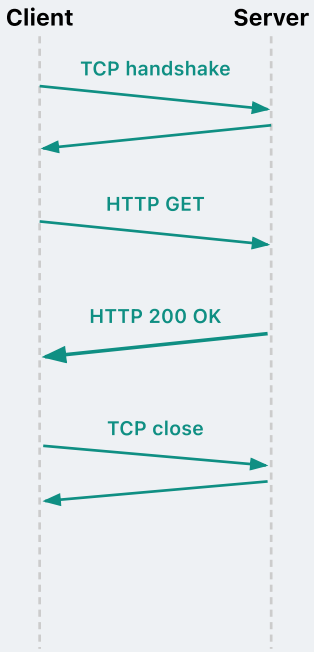
- **Multiple objects** can be sent over a single TCP connection
- HTTP/1.1, HTTP/2, and HTTP/3 use persistent HTTP

Protocol	Published	Connection type	Status
HTTP/1.0	1996	Non-persistent	Obsolete
HTTP/1.1	1997	Persistent	Standard
HTTP/2	2015	Persistent	Standard
HTTP/3	2022	Persistent (QUIC)	Standard

Non-persistent HTTP in action

Suppose a client wants to retrieve a file from: `http://www.someSchool.edu/someDept/pic.gif`

1. Client establishes a **TCP connection** with the server
2. Client sends an **HTTP GET** request — path: `/someDepartment/home.index`
3. Server retrieves the HTML file, wraps it in an **HTTP response**, and sends it back
4. Server signals TCP to **close the connection**



Response time: non-persistent HTTP

RTT | **Round Trip Time** — time for a small packet to travel from client to server and back

RESPONSE TIME | Time to retrieve a single object using HTTP

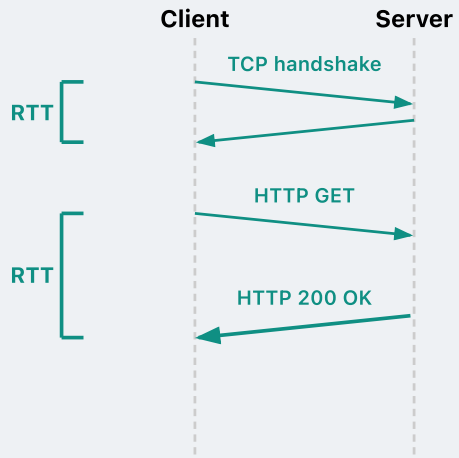
Q: What is the response time for non-persistent HTTP?

A: 2 RTTs + file transmission time

First RTT: the TCP handshake — client sends SYN, server replies SYN-ACK

Second RTT: client sends HTTP GET, server sends back the response

With many referenced objects, this cost multiplies — a significant burden on the server



Persistent HTTP in action

Client and server establish a TCP connection and exchange HTTP messages

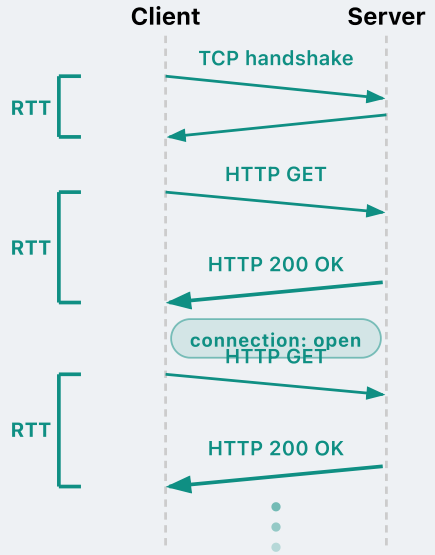
Server **keeps the TCP connection open** after sending the first HTTP response

Subsequent HTTP messages between the same client and server use the **same TCP connection** — no new handshake needed

Response time on one persistent connection:

1 RTT for initial TCP handshake + 1 RTT per HTTP request

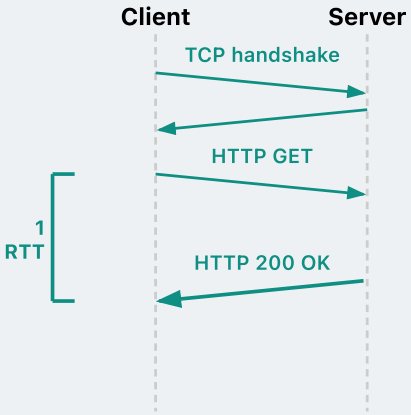
Amortised: **~1 RTT per object** for many requests



Improving HTTP efficiency further with pipelining

WITHOUT PIPELINING

Client issues a new request **only after** receiving the previous response — **1 RTT per file**



HTTP

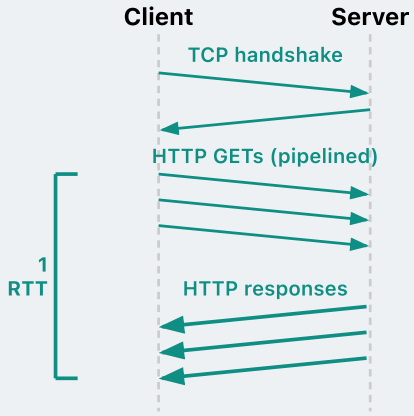
Improving HTTP efficiency further with pipelining

WITHOUT PIPELINING — Client issues a new request **only after** receiving the previous response — **1 RTT per file**

WITH PIPELINING — Client sends requests **as soon as it needs to retrieve an object**, without waiting for responses — e.g. in this example we pipeline 3 HTTP requests all together

Benefit: pipelining 3 requests → response time per file ≈ **1 RTT / 3**

- Don't confuse this with TCP-level pipelining
- HTTP pipelining operates at the **application layer** — it pipelines **HTTP messages**

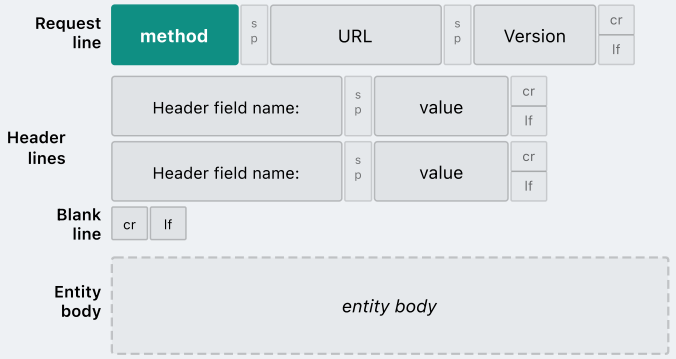


HTTP

HTTP request message format

Method — what action to perform

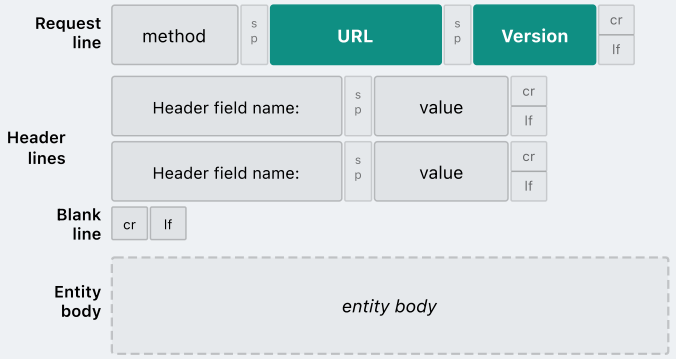
Method	Action
GET	**Retrieve a resource**
HEAD	Retrieve headers only (no body)
POST	**Submit data to a resource**
PUT	Store / replace a resource
DELETE	**Remove a resource**



HTTP

HTTP request message format

> **URL**: path to the requested resource — e.g. `/someDept/pic.gif`
> **HTTP Version**: identifies the protocol in use — `HTTP/1.0` or `HTTP/1.1`



HTTP

HTTP request message format

Header lines

- Zero or more `field-name: value` pairs, one per line
- Common examples:

```
Host: www.example.com
```

Mandatory in HTTP/1.1 — identifies the target server

```
Content-Type: text/html; charset=UTF-8
```

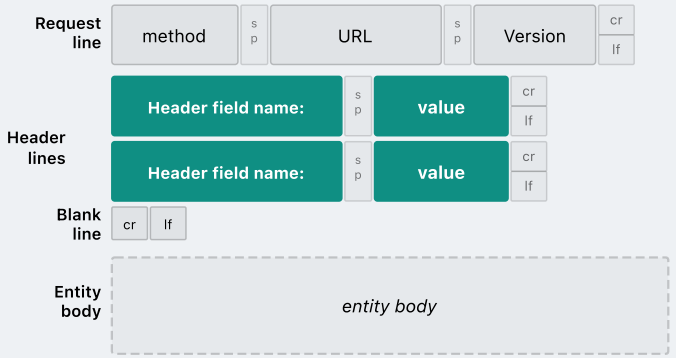
Format of the message body

```
Connection: close
```

Close the TCP connection after this response (non-persistent)

```
Accept-Language: da, en-gb;q=0.8, en;q=0.7
```

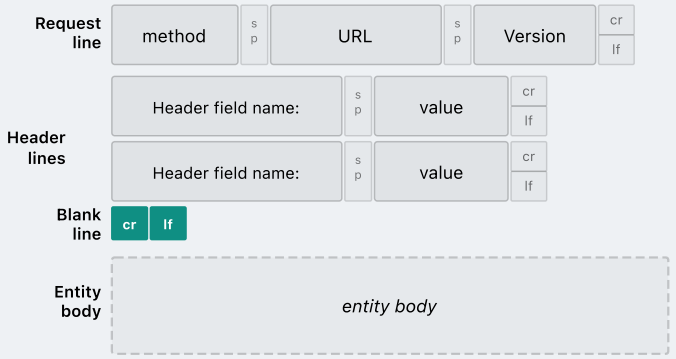
Language preference with quality weights (default q=1): prefer Danish → British English → any English



HTTP

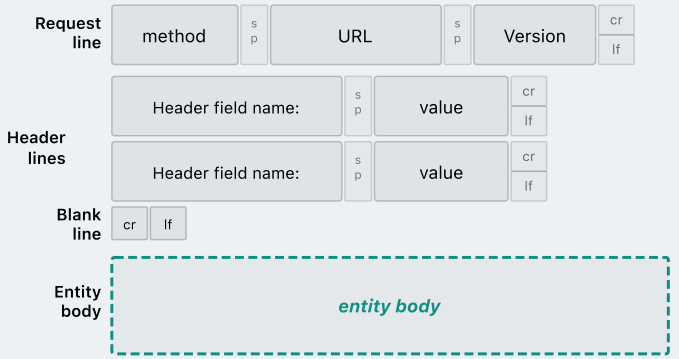
HTTP request message format

> **Blank line** (a lone CRLF): marks the end of the header section



HTTP request message format

> **Entity body**: empty for `GET` and `HEAD`; carries the payload for `POST` and `PUT` (e.g. form data or JSON)



HTTP request: an example

```
GET /somedir/page.html HTTP/1.1
Host: www.someschool.edu
User-agent: Mozilla/4.0
Connection: close
Accept-language: fr
---
(empty entity body)
```

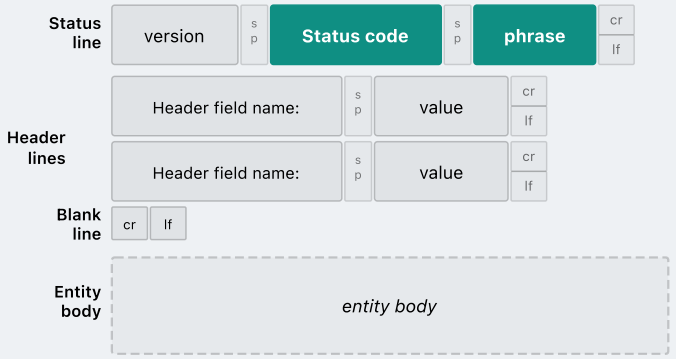
- **Method**: GET
- **URL**: /somedir/page.html
- **HTTP version**: HTTP/1.1
- **Header field**: Host: www.someschool.edu
- **Header field**: User-agent: Mozilla/4.0
- **Header field**: Connection: close
- **Header field**: Accept-language: fr
- **Body**: (empty)

HTTP

HTTP response message format

Status code & phrase

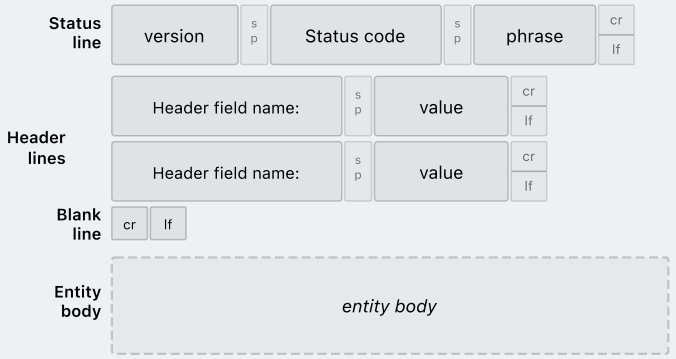
- **Status code**: 3-digit number indicating the result of the request
- **Phrase**: human-readable text accompanying the code — e.g. `OK`, `Not Found`



HTTP response message format

Common examples

Code	Phrase
200	OK
301	Moved Permanently
400	Bad Request
404	Not Found
505	HTTP Version Not Supported



HTTP response: an example

```
HTTP/1.1 200 OK
Connection: close
Date: Thu, 06 Aug 1998 12:00:15 GMT
Server: Apache/1.3.0 (Unix)
Last-Modified: Mon, 22 Jun 1998 …
Content-Length: 6821
Content-Type: text/html
---------------------------------------------------------
<!DOCTYPE html>
<html><head><title>Example</title></head>
<body><h1>Hello!</h1>
<script>console.log("hi");</script>
</body></html>
```

Status line: HTTP/1.1 200 OK

Body: the requested HTML file (with embedded JS)

HTTP

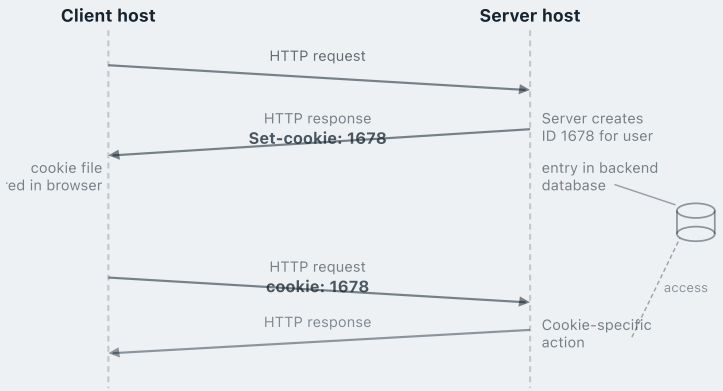
Cookies

> HTTP is **stateless** — every request is independent; the server retains no memory of past requests. But many applications *need* state. How does an online shopping site remember your cart, or recognise you across visits?

The answer: **cookies** — here's how it works.

1. A `Set-Cookie` header line in the **HTTP response** — the server sends the client a unique ID on first contact
2. A `Cookie` header line in subsequent **HTTP requests** — the client echoes the ID back on every future request
3. A **cookie file** stored on the client, managed by the browser — persists across sessions
4. A **back-end database** on the server, keyed by the cookie ID — stores per-user state

Cookies: an example [RFC 2109]

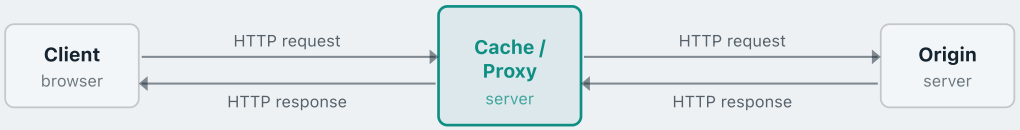


- The client sends an HTTP request to the host
- **Set-cookie: 1678** — Server assigns a unique ID and sends it to the client
- The cookie ID is stored as an entry in the back-end database, and in the client's browser
- **cookie: 1678** — Client attaches the ID on every future request to the same server
- Server uses the ID to look up the user's state in its database
- Server can then generate a **tailored response** — e.g. a personalised page, a pre-filled shopping cart — for the client's request

HTTP

Web caching

A **web cache** (also called a **proxy server**) is a network entity that satisfies HTTP requests on behalf of an origin server — storing copies of recently requested objects locally.



Acts as a server

Receives HTTP requests from clients and returns responses — just like an origin server would

Acts as a client

When the object is not cached, it issues its own HTTP request to the origin server to fetch it

HTTP

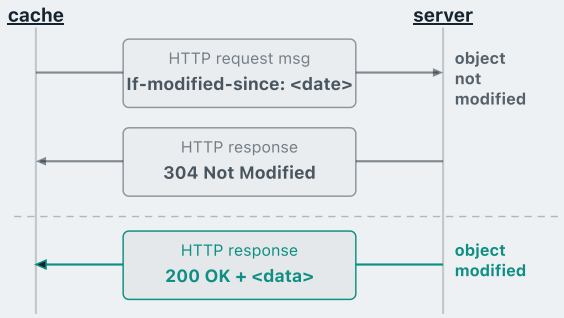
Why caching?

Faster response time
Cached objects are served from a nearby proxy — no round trip across the internet to the origin server. Substantially lower latency for the client.

Reduced access-link traffic
Requests satisfied by the cache never travel over the institution's uplink to the internet — less congestion on that bottleneck link.

Lower bandwidth costs
ISPs typically charge for outbound traffic volume. A cache that serves popular objects locally directly reduces that bill.

Conditional GET



Goal: the proxy server/cache wants to avoid re-fetching an object if its cached copy is still up-to-date

The proxy sends the following HTTP GET request:

```
GET /index.html HTTP/1.1
Host: www.example.com
If-modified-since: Mon, 10 Aug 2026 09:00:00 GMT
```

If the object is **unchanged**, server replies:

```
304 Not Modified
```

No object in the body — cache serves its stored copy directly

Object **changed**: server replies `200 OK` with the updated object in the body

MODULE 6

Email

Electronic Mail

1 · Network applications | 2 · The Web and HTTP | **3 · Email** | 4 · DNS

OUTLINE

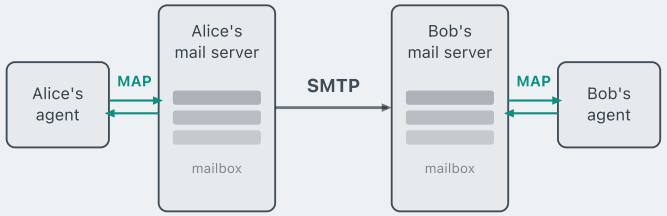
Email — roadmap

Electronic Mail (Email)

- *Mail access protocol:* **SMTP, POP3, IMAP**
- *Message format:* **MIME**

EMAIL

An overview of the email system



User agent (UA) — the app used to read, compose and send email (e.g. Outlook, Gmail, Apple Mail)

Mail server — a dedicated host that handles email on behalf of its users. It stores a **mailbox** per recipient for incoming mails and a **send queue** of outgoing mails

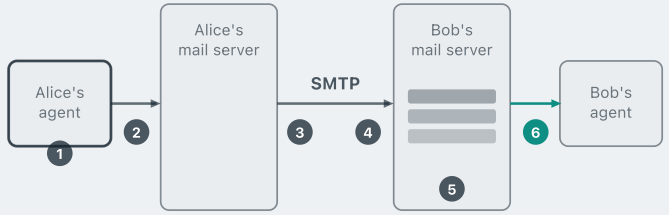
SMTP — Simple Mail Transfer Protocol; used between mail servers to transfer messages across the internet

Mail access protocol (MAP) — used by UAs to retrieve messages from their mail server; POP3 or IMAP

EMAIL · SMTP

SMTP in action

SMTP moves messages between mail servers. It consists of a **client side** and a **server side**.
Every mail server runs both an **SMTP client** (when sending) and an **SMTP server** (when receiving).



① Alice uses her UA to **compose** the message

② Alice's UA **sends** the message to her mail server

③ The **SMTP client** on Alice's server opens a TCP connection to Bob's mail server

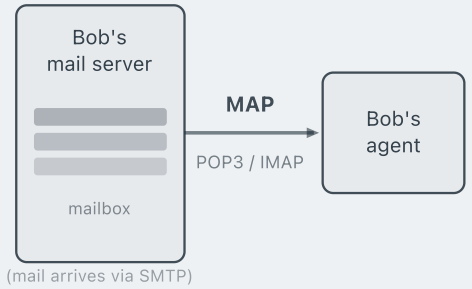
④ Alice's message is **transferred over the TCP connection**

⑤ The **SMTP server** on Bob's server receives the message and places it in Bob's **mailbox**

⑥ Bob invokes his UA to **read** the message

MAIL ACCESS PROTOCOL

Retrieving email from the server



Mail access protocols retrieve email from the mail server to the recipient's user agent

POP3 — Post Office Protocol version 3 [RFC 1939]
Downloads messages to the local device. Simple and stateless between sessions

IMAP — Internet Mail Access Protocol version 4 [RFC 3501]
Keeps messages on the server; supports folders and sync across multiple devices

WEB-BASED EMAIL

Accessing email via a browser

Web-based email is accessed through a web browser

Uses **HTTP** for communication between the user's browser and the mail server

Web-based email services

Examples: **Gmail**, **Outlook** (web)

IMAP

Internet Mail Access Protocol

IMAP **keeps messages on the server** and replicates copies to the client

Users can **organise messages in folders** — folder structure is stored on the server, accessible from any device

IMAP maintains **user state information** across sessions (e.g. folder names, message-to-folder mappings)

POP3

How POP3 works

UA establishes a **TCP connection** to the mail server on **port 110**	UA sends **username** and **password** (cleartext) to authenticate	UA **retrieves messages**; can mark for deletion, remove marks, obtain statistics	Server **deletes** marked messages and **closes** the connection

Notable limitations:

- Does not synchronise emails across multiple devices
- Does not support server-side folder management — users cannot organise emails into folders on the server

MESSAGE FORMAT

Structure of an email message

An email message is **ASCII text** — **header fields** followed (optionally) by a **body**

```
Date: Mon, 10 Aug 2026 09:00:00 GMT
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
Cc: charlie@example.com
```

Header lines

```
Hello Bob, here is the picture
you asked for. Enjoy!
```

Body

MIME

Multipurpose Internet Mail Extensions

Problem: standard email is ASCII-only — it cannot natively carry images, attachments, or non-ASCII characters (e.g. Chinese, Arabic)

MIME extends the message format with additional header fields that describe and encode non-ASCII content — telling the receiver how to decode the body

e.g. **base64** encoding converts binary data into printable ASCII text, which can be used for image attachments

```
From: alice@crepes.fr
To: bob@hamburger.edu
Subject: Picture of yummy crepe.
MIME-Version: 1.0
Content-Transfer-Encoding: base64
Content-Type: image/jpeg
---
(base64 encoded data…
…base64 encoded data)
```

MODULE 6

DNS

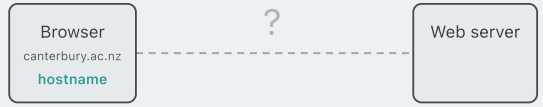
Domain Name System

1 · Network applications | 2 · The Web and HTTP | 3 · Email | **4 · DNS**

DNS

Domain Name System — overview

- As we have seen, network routing entirely relies on **IP addresses** — numerical identifiers
- However, when you browse the web, you typically type a **human-friendly hostname** such as `canterbury.ac.nz`
- How does the browser know how to translate between a hostname and an IP address?



DNS

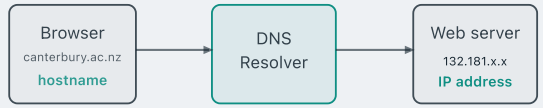
Domain Name System — overview

- As we have seen, network routing entirely relies on **IP addresses** — numerical identifiers
- However, when you browse the web, you typically type a **human-friendly hostname** such as `canterbury.ac.nz`
- How does the browser know how to translate between a hostname and an IP address?

> **DNS** is that translation mechanism — it maps human-readable **hostnames** to numerical **IP addresses**

> DNS implements a **distributed database** — a global hostname-to-IP mapping distributed across a hierarchy of DNS servers [RFC 1034, 1035]

> DNS is an **application-layer protocol** running over **UDP on port 53**



hostname → IP address

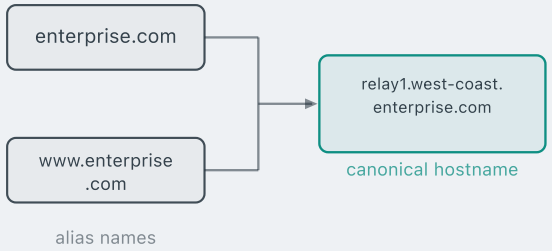
DNS SERVICES

Additional services provided by DNS

Hostname-to-IP translation — the primary service, already seen; commonly used by HTTP, SMTP, and other application-layer protocols

Host aliasing — a host with a complex **canonical hostname** can be reached via simpler **alias names**
e.g. `enterprise.com` and `www.enterprise.com` both resolve to the canonical `relay1.west-coast.enterprise.com`

Mail server aliasing — the same idea for email; a friendly address like `bob@outlook.com` can map to the canonical mail server hostname `relay1.west-coast.outlook.com`



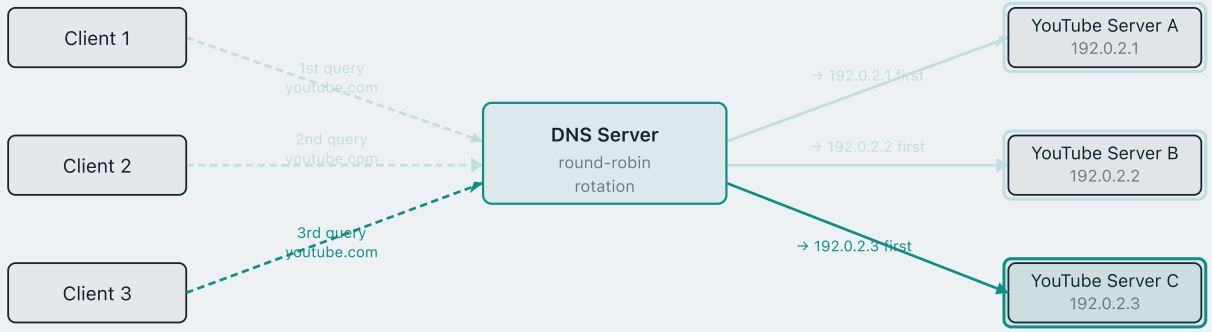
DNS SERVICES

Additional services by DNS: load distribution

Many popular web services — e.g. YouTube, Google, Netflix — are hit by **millions of users** simultaneously. A **single server** cannot handle that volume.

The service is therefore distributed across **many server instances**, each with a **distinct IP address**, all sharing the **same hostname** (e.g. `youtube.com`).

How are users assigned to these servers? **DNS load balancing** — DNS stores all the IPs and **rotates their order** on each response, so consecutive clients get a different IP listed first.



DNS

Why not a centralised design?

A centralised design would use a **single server** that hosts the translation database — every DNS query goes to that server.

Single point of failure

If the one server crashes, DNS lookup fails for the entire Internet

Traffic volume

Billions of DNS queries per day — no single machine can handle that load

Distant centralised database

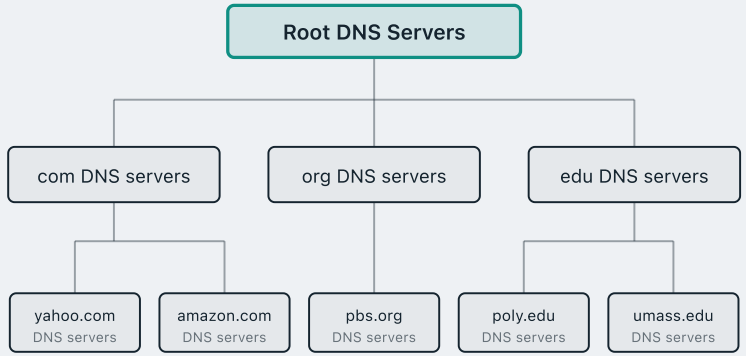
All queries must travel to one location — users far away face high latency on every lookup

Maintenance

Billions of hostname records change constantly — impossible to keep one database current

DNS

A hierarchical distributed database

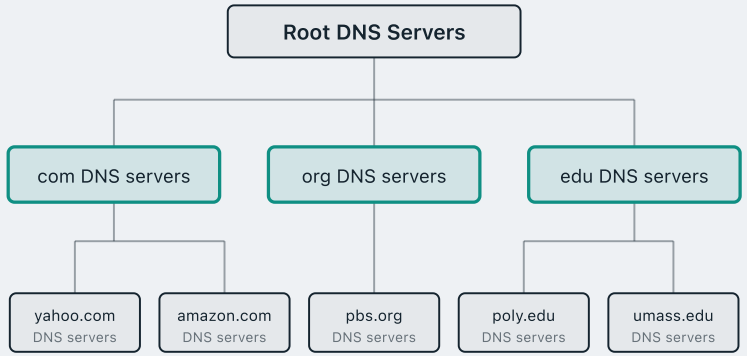


Root DNS Servers

13 root server addresses — each backed by a cluster of replicated machines, giving **over 1000 physical instances** worldwide

DNS

A hierarchical distributed database

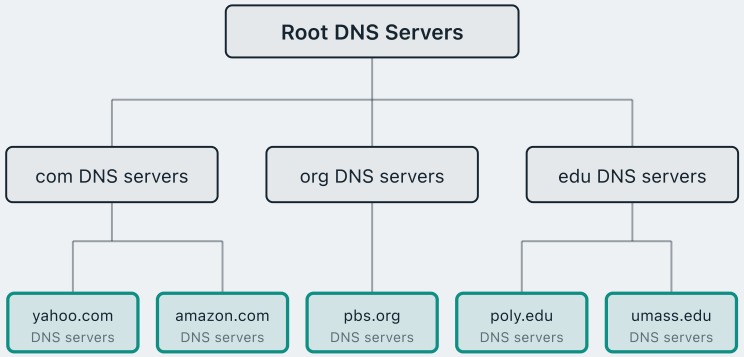


TLD DNS Servers

Handle **.com**, **.org**, **.edu** and country-code domains (.nz, .uk). Maintained by dedicated registries — e.g. **Verisign** (.com), **Public Interest Registry** (.org), **EDUCAUSE** (.edu), **InternetNZ** (.nz).

DNS

A hierarchical distributed database



Authoritative DNS Servers

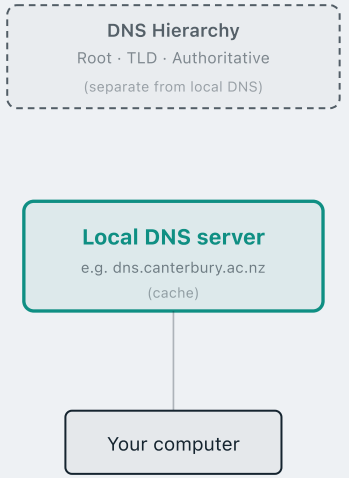
Each organisation's own DNS server — the **definitive source** for its hostname-to-IP mappings. e.g. Canterbury's authoritative server holds the IP for `www.canterbury.ac.nz`

DNS ARCHITECTURE

Local DNS server and DNS caching

Every ISP or institution operates a **local DNS server** (also called a *default name server*). It is **not** part of the root / TLD / authoritative hierarchy — it sits **separately** from that.

When you join the network, your device is automatically configured to use one — e.g. the University of Canterbury runs its own local DNS server for all campus users.



DNS ARCHITECTURE

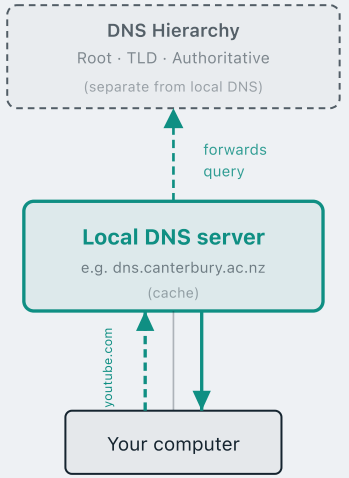
Local DNS server and DNS caching

When your machine makes a DNS lookup, the query goes **first to the local DNS server**.

If the answer is cached: the local DNS server replies **immediately** — no hierarchy lookup needed.

If not cached: the local server **forwards the query into the hierarchy**, relays the reply back to you, and **caches the hostname-to-IP mapping** in local memory for future queries.

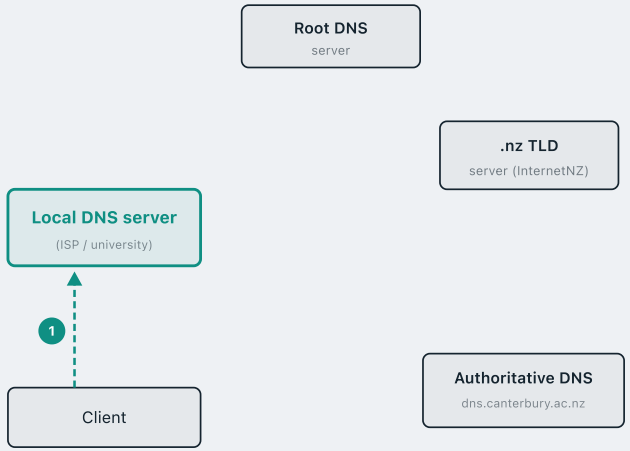
Cache entries have a **TTL (time-to-live)** and expire after some time (typically hours). TLD server addresses are routinely cached in local DNS servers — so **root servers are bypassed for most everyday queries**.



DNS ARCHITECTURE

Resolving a hostname: the full journey

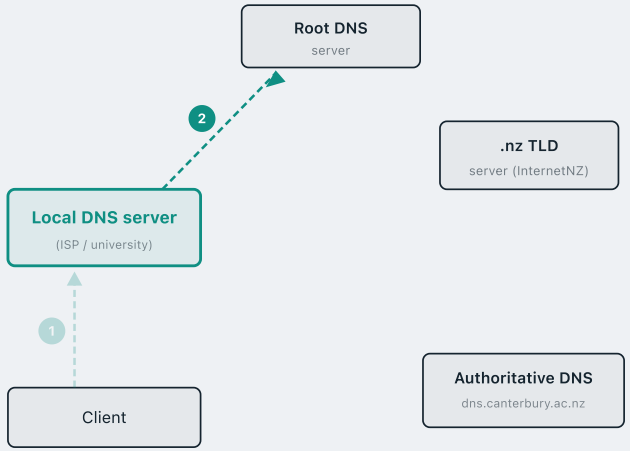
Step 1. A client wants to resolve **csse.canterbury.ac.nz**. It sends a DNS query to its **local DNS server**.



DNS ARCHITECTURE

Resolving a hostname: the full journey

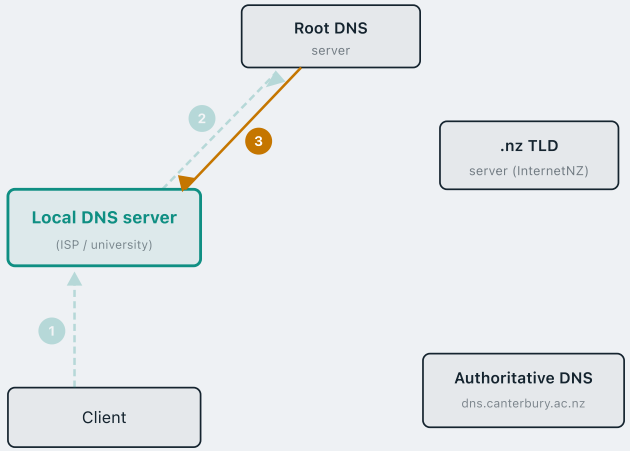
Step 2. The local DNS doesn't have the answer cached. It contacts a **root DNS server**.



DNS ARCHITECTURE

Resolving a hostname: the full journey

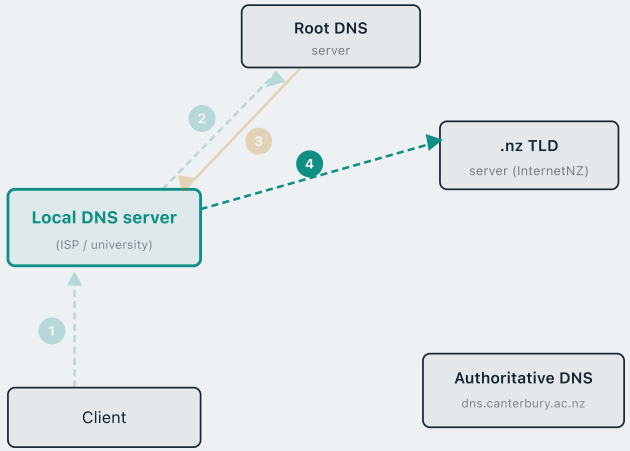
Step 3. The root server returns a **referral**: "I don't know — try a **.nz TLD** server" and gives its address.



DNS ARCHITECTURE

Resolving a hostname: the full journey

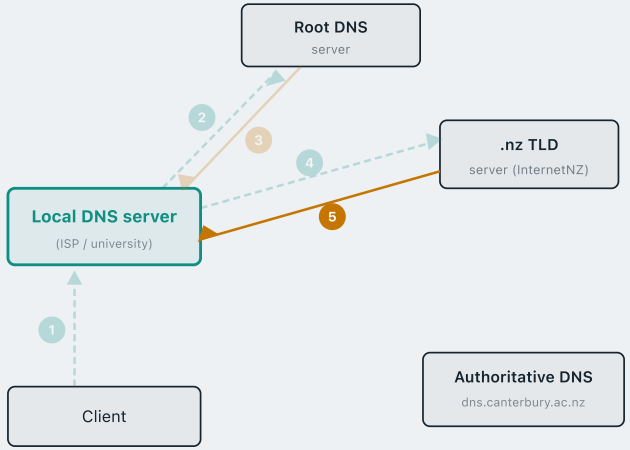
Step 4. The local DNS follows the referral and contacts the **.nz TLD server**.



DNS ARCHITECTURE

Resolving a hostname: the full journey

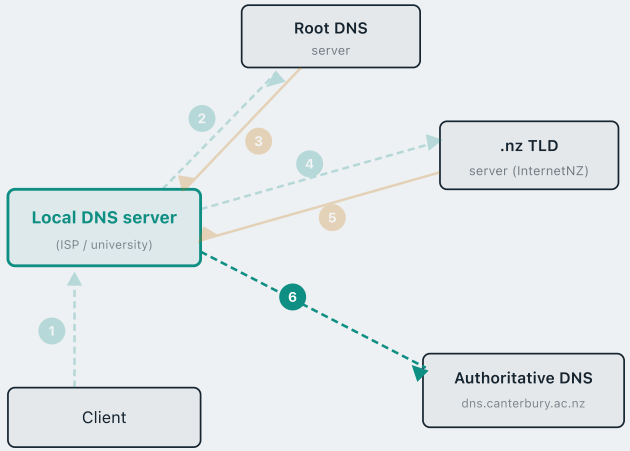
Step 5. The .nz TLD returns another **referral**: "Try **dns.canterbury.ac.nz** — the authoritative server for canterbury.ac.nz."



DNS ARCHITECTURE

Resolving a hostname: the full journey

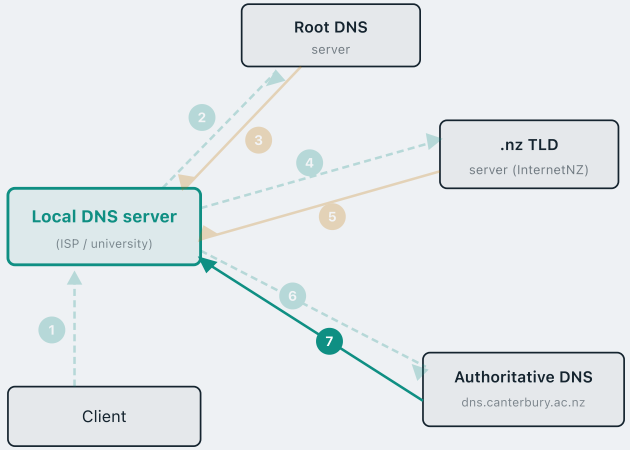
Step 6. The local DNS contacts **dns.canterbury.ac.nz** — the authoritative name server for the canterbury.ac.nz zone.



DNS ARCHITECTURE

Resolving a hostname: the full journey

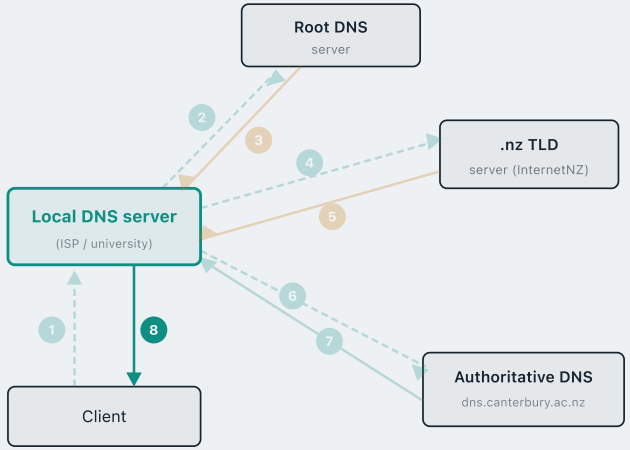
Step 7. The authoritative server has the answer! It replies: "csse.canterbury.ac.nz = **132.181.2.21**"



DNS ARCHITECTURE

Resolving a hostname: the full journey

Step 8. The local DNS returns **132.181.2.21** to the client — and **caches** the mapping for future queries.



DNS ARCHITECTURE

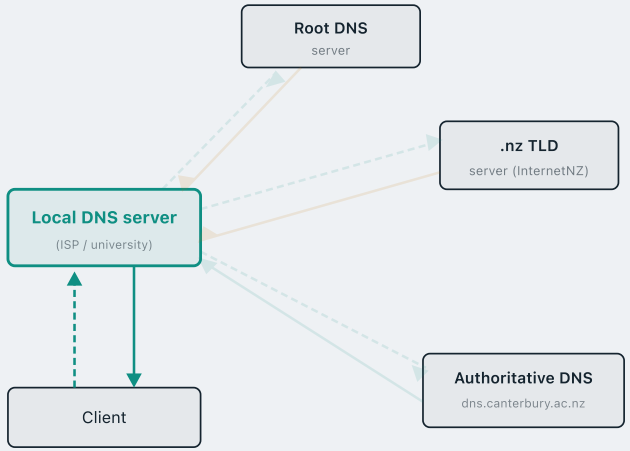
Recursive vs iterative queries

Iterative query

The contacted server does **not** resolve the name for you. It returns a **referral** — "I don't know, try that server."

Recursive query

The contacted server takes **full responsibility** — it resolves the name itself (querying the next server in the chain) and returns the **final answer**.



DNS ARCHITECTURE

Recursive vs iterative queries

Iterative query

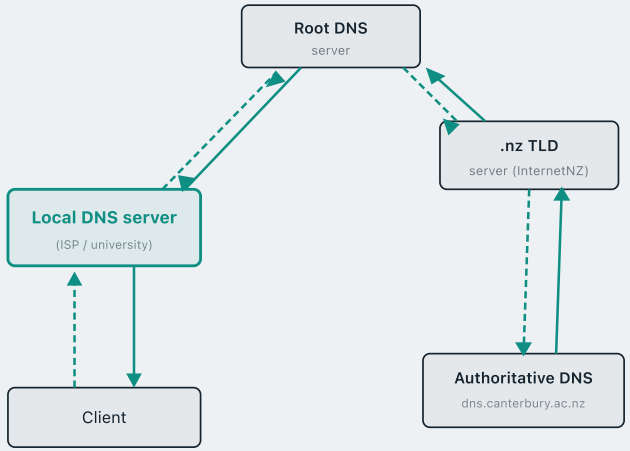
The contacted server does **not** resolve the name for you. It returns a **referral** — "I don't know, try that server."

Recursive query

The contacted server takes **full responsibility** — it resolves the name itself (querying the next server in the chain) and returns the **final answer**.

Can we go fully recursive?

Yes — instead of local DNS following referrals itself, it passes the query to root, which asks TLD, which asks auth. The answer cascades back through the chain.



DNS ARCHITECTURE

DNS Resource Records

DNS distributes data as **Resource Records (RRs)**. Each record has four fields:

`(name, value, type, TTL – Time-To-Live)`

NAME	VALUE	TYPE	TTL
web.canterbury.ac.nz	132.181.106.9	**A**	XXXX
itdns.canterbury.ac.nz	132.181.2.225	**A**	XXXX
mail.canterbury.ac.nz	132.181.3.225	**A**	XXXX
canterbury.ac.nz	itdns.canterbury.ac.nz	**NS**	XXXX
canterbury.ac.nz	web.canterbury.ac.nz	**CNAME**	XXXX
canterbury.ac.nz	mail.canterbury.ac.nz	**MX**	XXXX

DNS ARCHITECTURE

DNS Resource Records

Type A

name = hostname · **value** = IPv4 address

NAME	VALUE	TYPE	TTL
web.canterbury.ac.nz	132.181.106.9	A	XXXX
itdns.canterbury.ac.nz	132.181.2.225	A	XXXX
mail.canterbury.ac.nz	132.181.3.225	A	XXXX
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	XXXX
canterbury.ac.nz	web.canterbury.ac.nz	CNAME	XXXX
canterbury.ac.nz	mail.canterbury.ac.nz	MX	XXXX

DNS ARCHITECTURE

DNS Resource Records

Type NS

name = domain · **value** = hostname of the authoritative DNS server for that domain

"I don't know — but here's the address of the server that does."

NAME	VALUE	TYPE	TTL
web.canterbury.ac.nz	132.181.106.9	A	XXXX
itdns.canterbury.ac.nz	132.181.2.225	A	XXXX
mail.canterbury.ac.nz	132.181.3.225	A	XXXX
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	XXXX
canterbury.ac.nz	web.canterbury.ac.nz	CNAME	XXXX
canterbury.ac.nz	mail.canterbury.ac.nz	MX	XXXX

DNS ARCHITECTURE

DNS Resource Records

Type CNAME

name = alias hostname · **value** = canonical (real) hostname it maps to

NAME	VALUE	TYPE	TTL
web.canterbury.ac.nz	132.181.106.9	A	XXXX
itdns.canterbury.ac.nz	132.181.2.225	A	XXXX
mail.canterbury.ac.nz	132.181.3.225	A	XXXX
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	XXXX
canterbury.ac.nz	web.canterbury.ac.nz	CNAME	XXXX
canterbury.ac.nz	mail.canterbury.ac.nz	MX	XXXX

DNS Resource Records

Type MX

name = domain · **value** = hostname of the mail server for that domain

NAME	VALUE	TYPE	TTL
web.canterbury.ac.nz	132.181.106.9	A	XXXX
itdns.canterbury.ac.nz	132.181.2.225	A	XXXX
mail.canterbury.ac.nz	132.181.3.225	A	XXXX
canterbury.ac.nz	itdns.canterbury.ac.nz	NS	XXXX
canterbury.ac.nz	web.canterbury.ac.nz	CNAME	XXXX
canterbury.ac.nz	mail.canterbury.ac.nz	MX	XXXX

DNS ARCHITECTURE

DNS protocol message format

DNS uses the **same message format** for both **queries** and **replies** — a fixed 12-byte header followed by four variable-length sections.

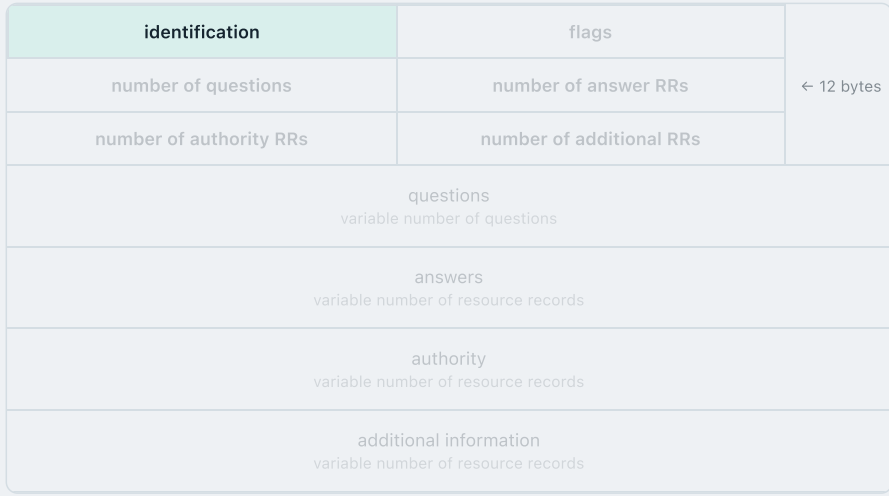
identification	**flags**	← 12 bytes
number of questions	**number of answer RRs**	
number of authority RRs	**number of additional RRs**	
questions variable number of questions		
answers variable number of resource records		
authority variable number of resource records		
additional information variable number of resource records		

DNS ARCHITECTURE

DNS protocol message format

Identification

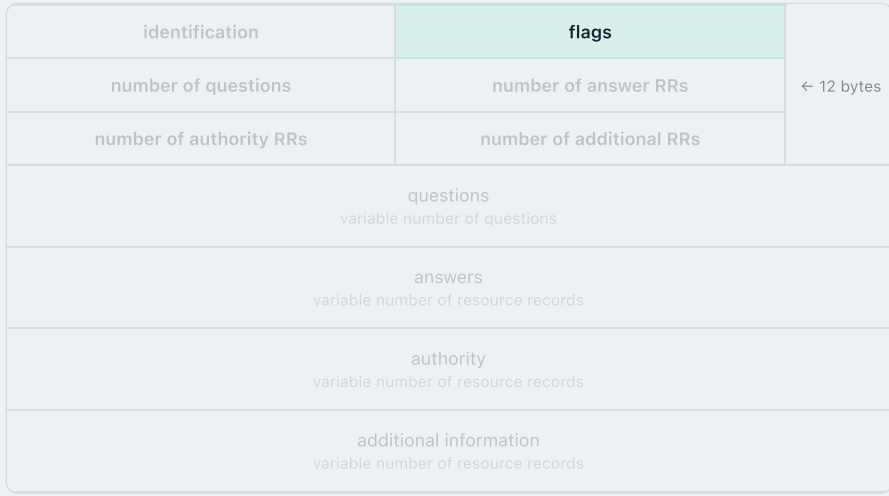
16-bit number assigned by the client. The reply copies this same value, so the client can match each reply to its original query.



DNS protocol message format

Flags

- Query (0) or reply (1)
- Recursion desired — client requests full recursive resolution
- Recursion available — server indicates it supports recursion

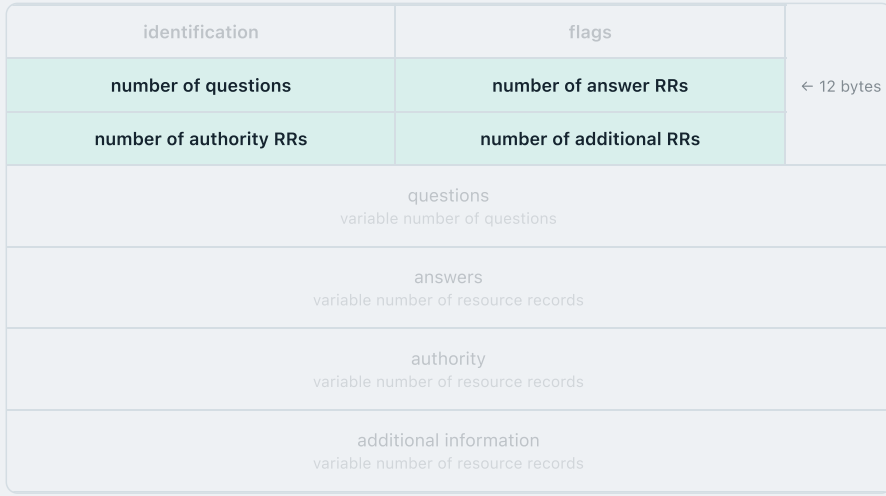


DNS ARCHITECTURE

DNS protocol message format

Count fields

Four 16-bit counters — one per section below — telling how many records appear in each variable-length section.



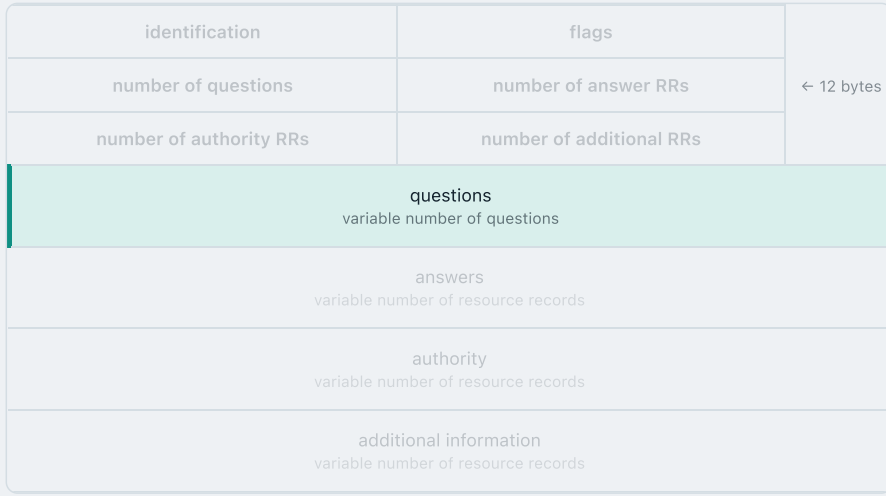
DNS ARCHITECTURE

DNS protocol message format

Questions

The name and record type being queried. e.g. this Type A question asks for the IP address of the domain facebook.com.

```
(facebook.com, A)
```



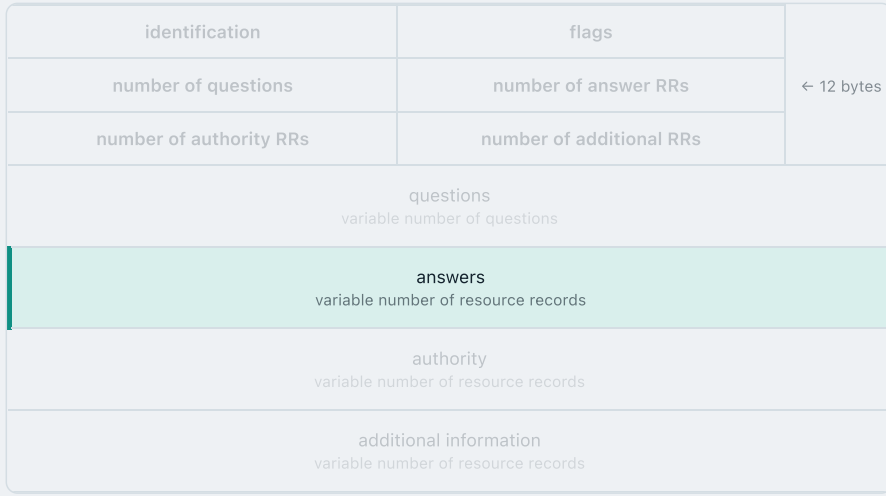
DNS ARCHITECTURE

DNS protocol message format

Answers

Resource Records (RRs) that directly answer the query.

```
(facebook.com, 157.240.22.35, A, TTL=300)
```



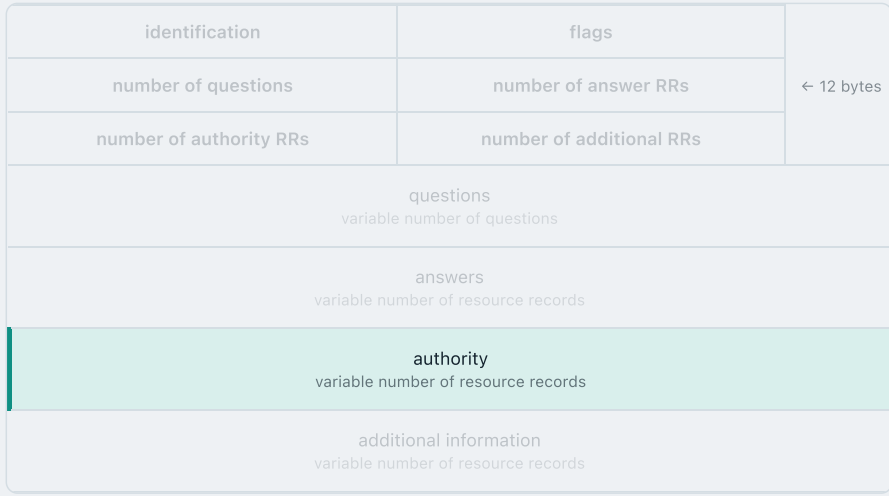
DNS ARCHITECTURE

DNS protocol message format

Authority

NS records pointing to authoritative servers — returned in referral responses.

```
(facebook.com, a.ns.facebook.com, NS)
```



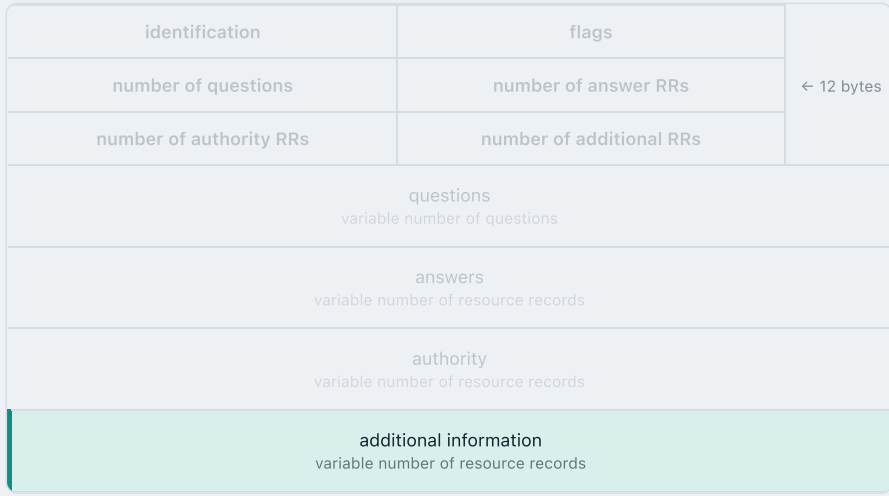
DNS ARCHITECTURE

DNS protocol message format

Additional

'Glue' Resource Records (RRs) accompanying the authority section — so you can reach the name server without an extra lookup.

```
(a.ns.facebook.com, 129.134.30.12, A)
```



DNS ARCHITECTURE

Query & response: a live example

QUERY →

Transaction ID	0x6492
Flags	0x0100 (query)
Questions	1
Answer RRs	0
Authority RRs	0
Additional RRs	0

QUESTIONS

www.facebook.com A

← RESPONSE

Transaction ID	0x6492
Flags	0x8180 (response, no error)
Questions	1
Answer RRs	2
Authority RRs	4
Additional RRs	8

QUESTIONS

www.facebook.com A

ANSWERS

www.facebook.com CNAME
star-mini.c10r.facebook.com

star-mini.c10r.facebook.com A
31.13.78.35

AUTHORITY

c10r.facebook.com NS
d.ns.c10r.facebook.com (+3)

ADDITIONAL

d.ns.c10r.facebook.com A
185.89.219.11 (+7)

DNS

DNS — key takeaways

- **Main purpose** — DNS maps human-readable hostnames to IP addresses.
- **Hierarchy** — DNS is distributed: no single server holds all records.
  Responsibility is delegated across Root, TLD, and Authoritative servers.
- **Traversal pattern** — the client queries the local DNS resolver, which then follows referrals Root → TLD → Authoritative.
  Results are cached locally by TTL to avoid repeated lookups.

References

James F. Kurose & Keith W. Ross, *Computer Networking: A Top-Down Approach*, Addison Wesley, 3rd edition, 2005.

Chapter 2 Application Layer (2.3, 2.4)

William Stallings, *Data and Computer Communications*, Pearson, 10th edition, 2013.

Chapter 24 Electronic Mail, DNS, and HTTP (24.1, 24.2)

Andrew S. Tanenbaum & David J. Wetherall, *Computer Networks*, Prentice Hall, 5th edition, 2010.

Chapter 7 The Application Layer (7.1, 7.2)

Acknowledgements

These lecture notes are developed based on slides and notes from:

Barry Wu's lecture notes for COSC264, University of Canterbury

Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024

Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017

Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005